Relatório Comparativo do SonarQube

Métricas de qualidade da Fase 02 e da Fase 03
Sistema de Batalhas PokeSal

As duas fases foram analisadas com a mesma instalação do SonarQube Community 26.11, rodando localmente em um container Docker, e enviadas pelo plugin do Maven (mvn clean verify sonar:sonar). A Fase 02 corresponde ao commit 49ac62e, o mesmo do relatório anterior, com 192 testes. A Fase 03 corresponde ao commit aeff114, com 290 testes. A cobertura foi medida pelo JaCoCo 0.8.14 e importada pelo SonarQube. Os números são da visão Overall Code, que considera o projeto inteiro.

A Fase 02 foi analisada de novo, em vez de reaproveitar o relatório anterior, porque a versão do SonarQube mudou (de 26.9 para 26.11). Nesta versão, o perfil padrão para Java passou a ser o Sonar way core, que tem só regras de bugs e vulnerabilidades. Para que os code smells continuassem sendo medidos, os dois projetos usaram o perfil Sonar way comprehensive, o único que contém todas as regras apontadas no relatório da Fase 02. Mesmo assim, a regra java:S5786 (classes de teste públicas), que gerou 10 ocorrências na análise anterior, não gerou nenhuma nesta versão, embora continue ativa no perfil. Por isso a Fase 02 aparece aqui com 37 code smells, e não com os 47 do relatório anterior.

Comparativo

Métrica	Fase 02	Fase 03	Variação
Bugs (Reliability)	0	0	sem alteração
Vulnerabilities (Security)	0	0	sem alteração
Code Smells (Maintainability)	37	52	+15 (piorou)
Coverage	71,3%	77,1%	+5,8 pontos (melhorou)
Duplications	0,0%	0,0%	sem alteração
Linhas de código	1.409	2.111	+702 (+49,8%)

Nesta versão do SonarQube, o painel agrupa as issues pela qualidade de software que afetam: Reliability corresponde aos Bugs, Security às Vulnerabilities e Maintainability aos Code Smells.

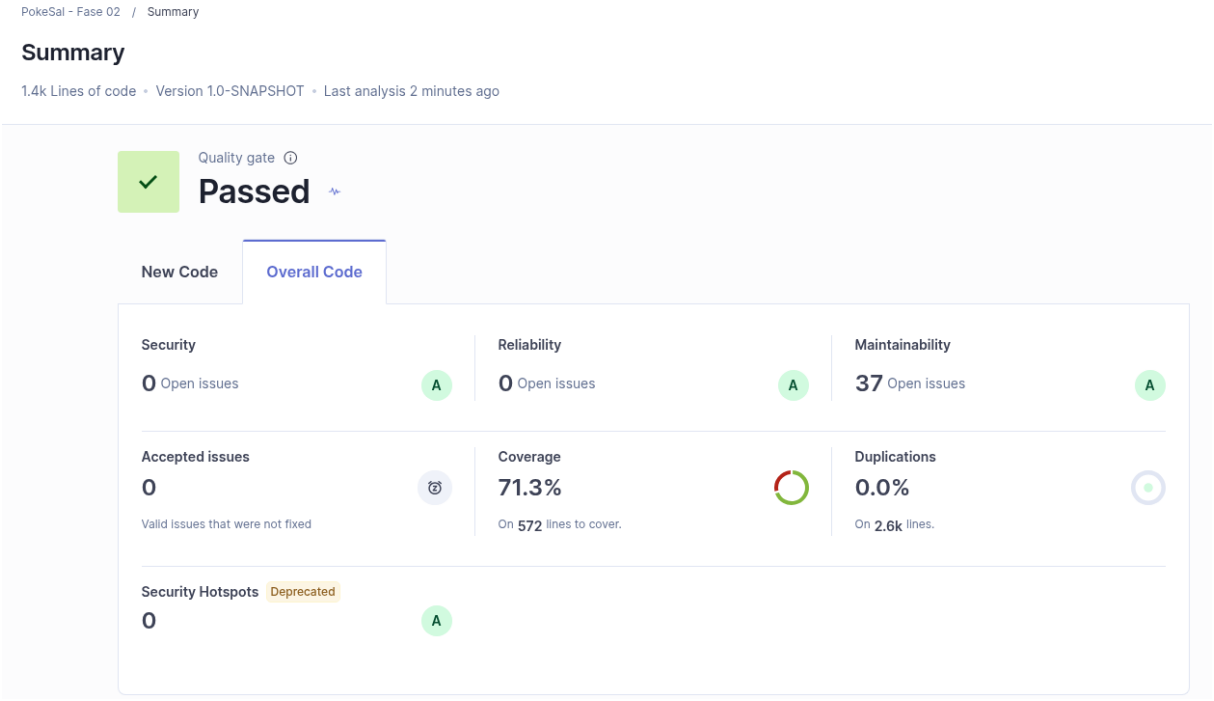


Figura 1 — Painel do projeto na Fase 02 (commit 49ac62e), visão Overall Code.

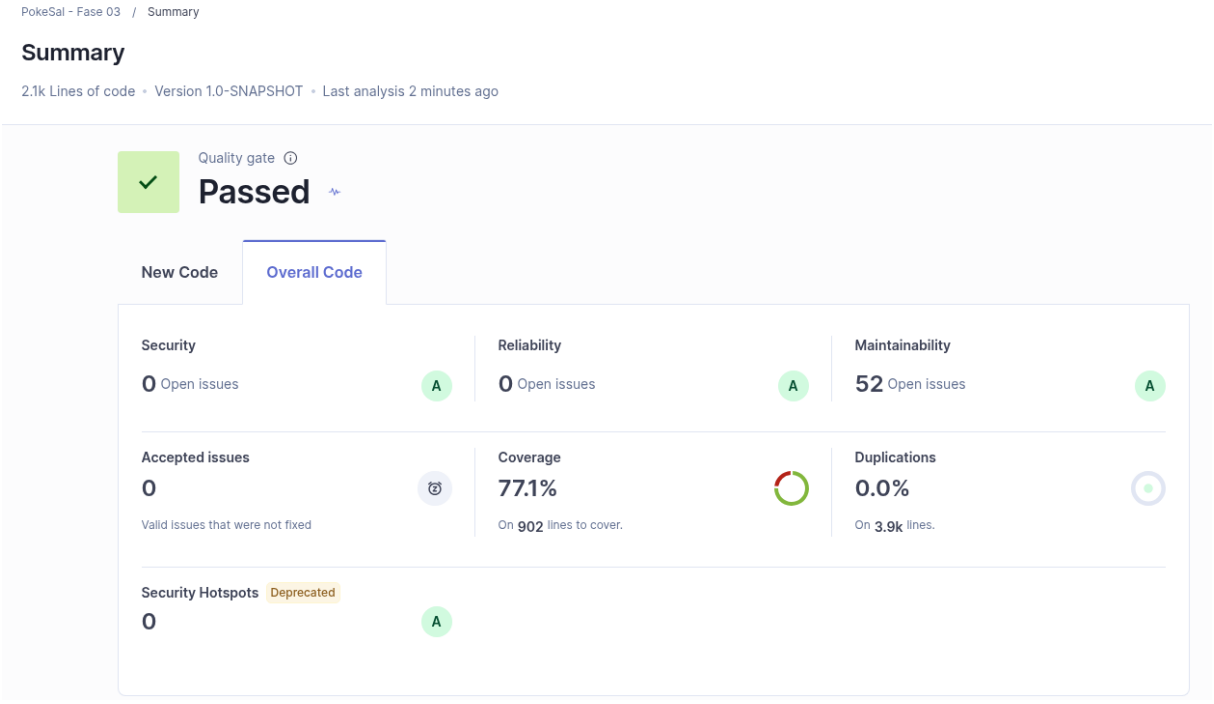


Figura 2 — Painel do projeto na Fase 03 (commit aeff114), visão Overall Code.

Code Smells: a métrica que piorou

Os code smells passaram de 37 para 52. Todos são de manutenibilidade, e a severidade se manteve parecida: 2 altas, 48 médias, 1 baixa e 1 informativa na Fase 03, contra 2, 33, 1 e 1 na Fase 02. A tabela mostra a variação por regra.

Regra	Descrição	Fase 02	Fase 03
java:S106	Uso de System.out no lugar de um logger	27	39
java:S5778	Lambda de assertThrows com mais de uma chamada que pode lançar exceção	4	6
java:S6126	Concatenação de texto que poderia ser um text block	0	1
java:S1192	Literais de texto repetidos	2	2
java:S107	Construtor com mais de 7 parâmetros	1	1
java:S1172	Parâmetro não usado	1	1
java:S7467	Variável de exceção não usada	1	1
java:S1135	Comentário TODO pendente	1	1

java:S106 — Uso de System.out (+12)

Onde: BattleCli (de 25 para 36) e BattleRunner (de 2 para 3). No BattleCli, as ocorrências novas vêm do torneio: o menu de escolha entre batalha simples e torneio, a pergunta sobre a quantidade de treinadores e os anúncios de cada confronto e do campeão. No BattleRunner, as duas mensagens de cadastro ("Treinador 1:" e "Treinador 2:") viraram uma só, dentro de um laço, e as duas ocorrências novas são as linhas em que o System.out é entregue ao ConsoleBattleLogger e ao ConsoleTrainerNotifier.

Justificativa: a avaliação é a mesma da Fase 02. O System.out é a própria interface do jogo no terminal, e trocá-lo por um logger mudaria o comportamento da aplicação. As classes de saída criadas na refatoração, ConsoleBattleLogger e ConsoleTrainerNotifier, recebem o PrintStream pelo construtor e não acessam o System.out diretamente: ele só é referenciado no BattleRunner, o ponto que monta as dependências. É o efeito esperado da inversão de dependência, e é o que permite testar essas classes com um PrintStream em memória.

java:S5778 — Lambda de assertThrows com mais de uma chamada (+2)

Onde: MoveTest, linha 55, em que a lambda cria o status Poison e o golpe na mesma expressão, e RunBattleUseCaseTest, linha 338, em que a lambda cria o caso de uso e chama o execute.

Justificativa: as duas ocorrências são válidas, como as quatro que já existiam na Fase 02. O risco é o teste passar porque a exceção veio da chamada errada. Nos dois casos, a exceção esperada só pode vir da chamada final, mas o ajuste é simples: criar o status e o caso de uso antes da lambda.

java:S6126 — Concatenação no lugar de text block (+1)

Onde: ConsoleBattleLoggerTest, linha 84, na mensagem esperada de duas linhas.

Justificativa: válida e de baixo impacto. Um text block deixaria a mensagem esperada mais legível.

Proporção ao tamanho do código

O código cresceu quase 50% na Fase 03 (de 1.409 para 2.111 linhas), com as novas interfaces, o torneio e os dois requisitos autorais. Considerando o tamanho, a densidade de code smells caiu de 26,3 para 24,6 por mil linhas de código. As regras que já apareciam na Fase 02 e foram avaliadas como aceitas ou falsos positivos (java:S1192, java:S1172 e java:S1135) continuam iguais.

Coverage

A cobertura geral subiu de 71,3% para 77,1%. O SonarQube calcula esse valor combinando linhas e condições, e as duas melhoraram: a cobertura de linhas foi de 70,3% para 72,6% e a de condições, de 74,3% para 90,6%.

A melhora nas linhas parece pequena por causa de uma mudança de configuração. Na Fase 03, o JaCoCo passou a medir apenas as classes de regra de negócio, deixando de fora o pacote io e o BattleRunner, que tratam da interação pelo terminal. Como o relatório do JaCoCo não traz dados desses arquivos, o SonarQube conta as 183 linhas deles como não cobertas, mesmo que o ConsoleBattleLogger e o ConsoleTrainerNotifier tenham ganhado testes. Comparando só as classes de regra de negócio nas duas fases, a evolução fica mais clara:

Classes de regra de negócio	Fase 02	Fase 03
Cobertura de linhas	83,2% (402 de 483)	91,1% (655 de 719)
Cobertura de condições	86,2% (150 de 174)	90,6% (269 de 297)

A Fase 03 atinge as metas de 80% de linhas e 70% de branches, e essa verificação faz parte do build: o mvn verify falha se a cobertura ficar abaixo desses valores.

Bugs, Vulnerabilities e Duplications

As três métricas continuaram zeradas. Os bugs encontrados durante a Fase 03 (BUG-07, o registro de status repetido, e BUG-08, as violações do Checkstyle) estão no registro de bugs e já estavam corrigidos quando a análise foi feita. Eram problemas de regra de negócio e de estilo, que as regras genéricas do SonarQube não apontam como bugs.

A duplicação continuou em 0,0%, mesmo com o crescimento do código. As partes repetidas da Fase 02 já tinham sido agrupadas em classes base, e as novidades da Fase 03 seguiram o mesmo caminho: cada colaborador externo tem uma única interface e uma única implementação.

Conclusão

Entre as cinco métricas pedidas, a única que piorou foi a quantidade de code smells, de 37 para 52. O aumento se explica quase todo pelo uso de System.out na interface de terminal, que cresceu com o modo torneio e já tinha sido avaliado como aceito na Fase 02, e por três ocorrências novas e válidas nos testes, de baixo impacto. Proporcionalmente ao tamanho do código, a densidade de code smells diminuiu. A cobertura melhorou, e bugs, vulnerabilidades e duplicações continuaram zerados.